



INSTITUTO FEDERAL DE SÃO PAULO
CAMPUS GUARULHOS
ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

MATHEUS LIMA DE SIQUEIRA - GU3065014
PEDRO DE CASTRO BARBA - GU3064263
RODRIGO DINIZ SOTERO - GU3064468
CAROLINE FREITAS MARCUSSO - GU3066266

ESTRUTURA DE DADOS - ALGORITMOS DE ORDENAÇÃO
Pesquisa acerca da descrição e histórico de criação do método de ordenação BucketSort

GUARULHOS – SP

1 - DESCRIÇÃO DO FUNCIONAMENTO DO ALGORITMO.....	2
2 - HISTÓRICO DE CRIAÇÃO.....	2
1. Contexto histórico e computacional.....	2
2. Problema que o algoritmo buscava resolver.....	3
3. Relação com outros algoritmos e evolução.....	3
4. Consolidação teórica.....	3
5. Considerações finais.....	3
3 - PASSO A PASSO DO CÓDIGO.....	4
1. Recebimento dos dados de entrada.....	4
2. Inicialização dos buckets.....	4
3. Distribuição dos elementos nos buckets.....	4
4. Ordenação interna dos buckets.....	4
5. Reunião dos elementos (concatenação).....	4
4 - COMPLEXIDADE COMPUTACIONAL.....	4
1. Complexidade do pior caso: $O(n^2)$	5
2. Complexidade do caso médio: $O(n + k)$	5
3. Complexidade do melhor caso: $O(n + k) \approx O(n)$	5
5 - CONCLUSÃO.....	6
6. BIBLIOGRAFIA.....	6

1 - DESCRIÇÃO DO FUNCIONAMENTO DO ALGORITMO

O Bucket Sort é um algoritmo de ordenação que se baseia na distribuição dos elementos de um conjunto em diferentes grupos, denominados buckets (ou recipientes). Inicialmente, os elementos de um vetor não ordenado são particionados em um número finito de buckets, de acordo com critérios específicos, geralmente relacionados ao intervalo de valores. Em seguida, cada bucket é ordenado individualmente, podendo-se utilizar outro algoritmo de ordenação adequado ou aplicar recursivamente o próprio método de Bucket Sort. Por fim, os elementos são reunidos a partir da concatenação dos buckets já ordenados, resultando em um vetor final ordenado.

Esse método apresenta melhor desempenho quando os dados estão uniformemente distribuídos, pois, nesse caso, os elementos tendem a se dividir de maneira equilibrada entre os buckets. Sua complexidade média pode atingir $O(n)$, dependendo da distribuição dos dados e do algoritmo utilizado na ordenação interna de cada recipiente. No entanto, em cenários desfavoráveis, sua complexidade pode se aproximar de $O(n^2)$, especialmente quando há concentração de muitos elementos em poucos buckets.

De acordo com a literatura, o Bucket Sort é especialmente eficiente para ordenar números reais distribuídos em intervalos conhecidos, sendo amplamente utilizado em situações nas quais se pode assumir uma distribuição aproximadamente uniforme dos dados.

2 - HISTÓRICO DE CRIAÇÃO

O algoritmo Bucket Sort não possui uma autoria atribuída a um único pesquisador ou a um momento específico da história da computação. De acordo com fontes especializadas, trata-se de um método que emergiu gradualmente no contexto do desenvolvimento dos primeiros algoritmos de ordenação, sendo consolidado ao longo das décadas como parte da família dos algoritmos de ordenação por distribuição.

1. Contexto histórico e computacional

Durante as décadas de 1950 a 1970, com o avanço dos computadores e o crescimento do volume de dados a serem processados, surgiu a necessidade de desenvolver algoritmos de ordenação mais eficientes. Nesse período, muitos algoritmos clássicos foram criados, como *Merge Sort* e *Quick Sort*.

Entretanto, esses métodos baseavam-se fortemente em comparações entre elementos, o que impunha um limite inferior de complexidade $O(n \log n)$ para a ordenação em muitos casos.

Nesse contexto, surgiram os chamados algoritmos de ordenação por distribuição, como o Bucket Sort, que buscavam contornar essa limitação ao evitar comparações diretas entre todos os elementos. Em vez disso, esses algoritmos exploram propriedades dos dados, como intervalo e distribuição dos valores.

2. Problema que o algoritmo buscava resolver

O principal problema que motivou o desenvolvimento do Bucket Sort foi a busca por algoritmos de ordenação mais rápidos que os métodos baseados em comparação, especialmente para conjuntos de dados com características específicas.

Mais precisamente, o objetivo era:

- Reduzir o custo computacional da ordenação;
- Explorar casos em que os dados estão uniformemente distribuídos;
- Permitir ordenação em tempo linear $O(n)$ em cenários favoráveis.

A ideia central consiste em dividir o problema maior em vários subproblemas menores (os buckets), tornando a ordenação mais eficiente ao trabalhar com subconjuntos reduzidos de dados.

3. Relação com outros algoritmos e evolução

O Bucket Sort está diretamente relacionado a outros algoritmos da mesma família, como:

- *Counting Sort*
- *Radix Sort*

Ele pode ser entendido como uma generalização do chamado Pigeonhole Sort (ou ordenação por caixas), permitindo múltiplos elementos por bucket, o que o torna mais flexível e aplicável a diferentes cenários.

Além disso, uma variação conhecida como Postman's Sort foi utilizada em sistemas reais, como máquinas de triagem postal, nas quais cartas são organizadas em etapas sucessivas (país, estado, cidade, etc.). Esse exemplo ilustra bem a ideia fundamental do algoritmo: dividir para organizar.

4. Consolidação teórica

O Bucket Sort foi posteriormente formalizado e amplamente difundido em obras clássicas da ciência da computação, como o livro *Algoritmos: Teoria e Prática*, onde é apresentado como um algoritmo eficiente sob determinadas condições, especialmente quando os dados seguem distribuição uniforme.

A formalização do algoritmo também reforçou sua análise de complexidade e suas limitações, destacando que seu desempenho depende diretamente da forma como os dados estão distribuídos.

5. Considerações finais

Em essência, o algoritmo Bucket Sort não é apenas mais um método de ordenação — é como organizar uma gaveta bagunçada em compartimentos organizados, facilitando a localização de tudo. Ao dividir os dados em grupos menores, ele simplifica o processo e aumenta o desempenho. Embora não seja uma solução universal, saber quando e como usá-lo pode economizar tempo e esforço.

3 - PASSO A PASSO DO CÓDIGO

O funcionamento do algoritmo Bucket Sort pode ser descrito de forma estruturada por meio das seguintes etapas:

1. Recebimento dos dados de entrada

Inicialmente, o algoritmo recebe um vetor contendo os elementos a serem ordenados.

2. Inicialização dos buckets

Define-se a quantidade de buckets (k), geralmente com base no número de elementos do vetor. Em seguida, são criados k recipientes vazios, que podem ser representados como listas ou arrays, destinados a armazenar temporariamente os elementos.

3. Distribuição dos elementos nos buckets

Cada elemento do vetor é alocado em um bucket específico, conforme uma função de mapeamento. Uma abordagem comum consiste na utilização da expressão:

Índice: $\text{valor} \times k / \text{valor máximo} + 1$

Em casos que envolvem números decimais, pode-se aplicar uma transformação prévia, como a multiplicação por um fator (por exemplo, 10), convertendo os valores em inteiros. Dessa forma, elementos com valores próximos, como 0,23 e 0,25, tendem a ser agrupados no mesmo bucket, pois compartilham a mesma parte inteira após a transformação.

4. Ordenação interna dos buckets

Após a distribuição, cada bucket não vazio é ordenado individualmente. Para isso, pode-se empregar algoritmos de ordenação simples, como Bubble Sort ou Insertion Sort, que apresentam bom desempenho para pequenas quantidades de elementos.

5. Reunião dos elementos (concatenação)

Por fim, os elementos são coletados sequencialmente a partir dos buckets, respeitando sua ordem e a ordenação interna de cada um. Esses elementos são reinseridos no vetor original, resultando em um conjunto completamente ordenado. Em implementações com alocação dinâmica, os elementos podem ser removidos dos buckets após a inserção, liberando memória ao longo do processo.

4 - COMPLEXIDADE COMPUTACIONAL

A análise da complexidade computacional do algoritmo Bucket Sort pode ser descrita conforme os diferentes cenários de execução, considerando a distribuição dos elementos e o método de ordenação aplicado em cada bucket.

1. Complexidade do pior caso: $O(n^2)$

O pior caso ocorre quando há uma distribuição inadequada dos dados, ou seja, quando muitos elementos possuem valores próximos e são alocados em um mesmo bucket. Essa situação provoca um desbalanceamento, no qual alguns buckets concentram uma grande quantidade de elementos, enquanto outros permanecem vazios ou subutilizados.

Nessa condição, o desempenho do algoritmo passa a depender diretamente do método de ordenação utilizado internamente nos buckets. Caso seja utilizado um algoritmo como o Insertion Sort, a complexidade da ordenação dentro de um bucket pode atingir $O(n^2)$, especialmente quando os elementos estão em ordem desfavorável.

O cenário mais crítico ocorre quando todos os elementos são inseridos em um único bucket, fazendo com que o Bucket Sort se comporte essencialmente como o algoritmo de ordenação interna adotado. Nesse caso, a complexidade total do algoritmo torna-se $O(n^2)$.

2. Complexidade do caso médio: $O(n + k)$

No caso médio, assume-se que os elementos estão distribuídos de forma aproximadamente uniforme entre os buckets, ainda que não perfeitamente equilibrada. Nessas condições, o Bucket Sort apresenta desempenho próximo ao linear, desde que não haja concentração excessiva de elementos em poucos buckets.

A complexidade total é composta por: distribuição dos elementos nos buckets $O(n)$, ordenação interna dos buckets: proporcional ao tamanho dos buckets

Assim, a complexidade é dada por $O(n + k)$, onde: n representa o número de elementos e k representa o número de buckets

Quando k é proporcional a n ($k = O(n)$), a complexidade pode ser simplificada para $O(n)$.

3. Complexidade do melhor caso: $O(n + k) \approx O(n)$

O melhor caso ocorre quando os elementos estão uniformemente distribuídos entre os buckets, de modo que cada bucket contenha uma quantidade pequena e aproximadamente igual de elementos.

Nessa situação a distribuição dos elementos ocorre em tempo $O(n)$, cada bucket contém poucos elementos (idealmente 0 ou 1) e a ordenação interna torna-se trivial, com custo constante ou desprezível

Assim, a complexidade total do algoritmo é $O(n + k)$. Quando o número de buckets é proporcional ao número de elementos ($k \approx n$), essa complexidade se aproxima de $O(n)$, caracterizando um desempenho linear.

5 - CONCLUSÃO

Diante do exposto, conclui-se que o algoritmo Bucket Sort constitui uma abordagem eficiente e conceitualmente simples para a ordenação de dados, fundamentada na estratégia de divisão do problema em subconjuntos menores. Ao distribuir os elementos em diferentes buckets e realizar a ordenação local desses grupos, o método reduz a complexidade do processo em cenários favoráveis, especialmente quando há distribuição uniforme dos dados.

Essa lógica pode ser comparada à organização de itens em compartimentos distintos, o que facilita não apenas a ordenação, mas também a compreensão do funcionamento do algoritmo. Tal característica evidencia a importância de estratégias que exploram propriedades específicas dos dados, em contraste com métodos puramente baseados em comparação.

Entretanto, destaca-se que o Bucket Sort não se apresenta como uma solução universal, uma vez que seu desempenho está diretamente condicionado à forma de distribuição dos elementos e à escolha do algoritmo de ordenação aplicado em cada bucket. Em situações desfavoráveis, sua eficiência pode ser significativamente reduzida.

Portanto, o conhecimento acerca de seu funcionamento, suas vantagens e limitações é essencial para sua aplicação adequada. Quando empregado em contextos apropriados, o Bucket Sort pode proporcionar ganhos relevantes de desempenho, otimizando o processamento e a organização de dados.

6. BIBLIOGRAFIA

- PROGRAMIZ. Bucket Sort. Disponível em: <https://www.programiz.com/dsa/bucket-sort>
. Acesso em: 24 mar. 2026.
- BUCKET sort. In: WIKIPÉDIA: a enciclopédia livre. Disponível em:
https://pt.wikipedia.org/wiki/Bucket_sort
. Acesso em: 24 mar. 2026.
- THE KNOWLEDGE ACADEMY. Bucket Sort Explained. Disponível em:
<https://www.theknowledgeacademy.com/blog/bucket-sort/>
. Acesso em: 24 mar. 2026.
- CORMEN, Thomas H. et al. Algoritmos: teoria e prática. 3. ed. Rio de Janeiro: Elsevier, 2012.
- GOODRICH, Michael T.; TAMASSIA, Roberto; GOLDWASSER, Michael H. Estruturas de dados e algoritmos em Java. 5. ed. Porto Alegre: Bookman, 2013.
- SZWARCFITER, Jayme Luiz; MARKENZON, Lilian. Estruturas de dados e seus algoritmos. 3. ed. Rio de Janeiro: LTC, 2010.
- BUCKET SORT. AlgoWalker. Disponível em: <https://www.algowalker.com/bucket-sort.html>

. Acesso em: 24 mar. 2026.

- BLACK, Paul E. Bucket sort. National Institute of Standards and Technology (NIST). Disponível em: <https://xlinux.nist.gov/dads/HTML/bucketsort.html>

. Acesso em: 24 mar. 2026.

